

In the name of God



University of Tehran

Faculty of Electrical and Computer Engineering

Deep Generative Models

Homework 2

Full Name	Mohammad Taha Majlesi Koupaei
Student ID	810101504
Submission Date	November 28, 2024

Contents

Contents	1
I Theoretical Questions	3
1 Question 1 - Diffusion Model	3
1.1 Mathematical Proof of Not Needing to Add Extra Noise in Denoising Diffusion Probabilistic Models (DDPM)	3
1.1.1 1. Representation of the distribution x_t	3
1.1.2 2. Independence of Noises	3
1.1.3 3. Reaching Intermediate Reconstructions	4
1.1.4 4. Using the Property of Sum of Normals	4
1.1.5 5. Conclusion	4
1.2 Sub-question 2: Conditions for Gaussian Reverse Process	4
1.2.1 Background	4
1.2.2 Conditions for $q(x_{t-1} x_t)$ to be Gaussian	5
1.2.3 Approximating the Reverse Path	5
1.3 Sub-question 3: DDPM Objective Function	5
1.3.1 Why are some terms simplified or ignored in practice?	6
1.3.2 Conclusion	7
1.4 Sub-question 4: Conditions for DDIM and DDPM Equivalence	7
1.4.1 Mathematical Explanation	7
1.4.2 Conditions for Equivalence	7
1.5 Sub-question 5: Early-Stopped DDPM (ES-DDPM)	8
1.5.1 The Core Idea	8
1.5.2 The Proposed Solution	8
1.5.3 Advantages of this Method	8
II Implementation Questions	10
2 DDPM and DDIM Implementation Analysis	10
2.1 Visual Evolution Through Epochs	10
2.1.1 Epoch 2	10
2.1.2 Epoch 20	10
2.1.3 Epoch 40 (Final)	11
2.2 Final Generated Samples	12
2.2.1 DDPM Final Samples	12
2.2.2 DDIM Final Samples	14
2.2.3 DDIM Deterministic Samples ($\eta = 0$)	16
2.3 Training Analysis and Diagnostics	18
2.3.1 Loss and Quality Analysis	18
2.3.2 Loss and FID Charts	18
2.3.3 FID Chart Analysis	20
2.4 Question 6: DDIM Parameter Ablation Study	20
2.4.1 Case 1: Fast Sampling ('eta=0.0', 'n_steps = 400')	20
2.4.2 Case 2: Quality-Focused Sampling ('eta=1', 'n_steps = 600')	21

2.5	Question 4: The Role of Noise ‘z’ in DDPM	21
2.5.1	Role of Noise ‘z’ in the Sampling Process	21
2.5.2	Why is Noise ‘z’ Zero at t=0?	22
3	Score-Based Models: Theory and Implementation	23
3.1	Theoretical Foundations	23
3.1.1	Sub-question 1: The Partition Function Problem	23
3.1.2	Sub-question 2: Sliced Score Matching (SSM)	23
3.1.3	Sub-question 3: Denoising Score Matching (DSM)	23
3.1.4	Sub-question 4: Langevin Dynamics Sampling	24
3.1.5	Sub-question 5: Annealed Langevin Dynamics	24
3.2	Implementation and Analysis	24
3.2.1	Data Visualization	24
3.2.2	Analysis of Score Fields with Fixed Sigma	25
3.2.3	Analysis of Sampling Trajectories	27
3.2.4	Final Sampling Results and Annealing	27
3.2.5	Annealed Sampling Implementation and Results	28
	References	30

Part I

Theoretical Questions

1 Question 1 - Diffusion Model

1.1 Mathematical Proof of Not Needing to Add Extra Noise in Denoising Diffusion Probabilistic Models (DDPM)

In the Denoising Diffusion Probabilistic Models (DDPM) paper, it is stated that adding extra noise during the forward diffusion process is not necessary, and intermediate reconstructions can be obtained without adding additional noise. To prove this claim, we use the property of the Gaussian distribution, which states that the sum of two independent random variables with normal distributions also has a normal distribution.

We have the following initial assumptions for this question:

- Let x_0 be the original data (without noise).
- The diffusion process is a Markov chain with T steps, where at each step $t = 1, 2, \dots, T$, the value x_t is obtained by adding Gaussian noise to x_{t-1} (the image from the previous step):

$$x_t = \sqrt{\alpha_t}x_{t-1} + \sqrt{1 - \alpha_t}\epsilon_t$$

where $\alpha_t = 1 - \beta_t$ and $\epsilon_t \sim \mathcal{N}(0, I)$.

Now, we will prove that adding noise in the forward path is not essential.

1.1.1 1. Representation of the distribution x_t

By recursively expanding, we can show that x_t can be expressed directly in terms of x_0 :

$$x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon$$

where:

$$\bar{\alpha}_t = \prod_{s=1}^t \alpha_s \quad \text{and} \quad \epsilon \sim \mathcal{N}(0, I)$$

This means that we can get from x_0 to x_t by adding a single, appropriately scaled Gaussian noise, rather than iterating through t steps.

1.1.2 2. Independence of Noises

We know that the noises $\epsilon_1, \epsilon_2, \dots, \epsilon_t$ are independent of each other at each step. Therefore, their weighted sum will also have a Gaussian distribution. The reparameterization trick shows that the cumulative noise is also a standard Gaussian.

1.1.3 3. Reaching Intermediate Reconstructions

Now we want to show that for $t' < t$, we can reach $x_{t'}$ from x_t without adding extra noise. Using the properties of the Gaussian distribution, we know that x_t can be expressed in terms of $x_{t'}$:

$$x_t = \sqrt{\frac{\bar{\alpha}_t}{\bar{\alpha}_{t'}}} x_{t'} + \sqrt{1 - \frac{\bar{\alpha}_t}{\bar{\alpha}_{t'}}} \epsilon'$$

where $\epsilon' \sim \mathcal{N}(0, I)$ is independent of the noise that formed $x_{t'}$. This equation shows a direct path from a less noisy state $x_{t'}$ to a more noisy state x_t .

1.1.4 4. Using the Property of Sum of Normals

Since $x_{t'}$ is also a linear combination of x_0 and Gaussian noise:

$$x_{t'} = \sqrt{\bar{\alpha}_{t'}} x_0 + \sqrt{1 - \bar{\alpha}_{t'}} \epsilon_{t'}$$

By substituting this into the equation for x_t , we can verify the consistency of the process. This rewriting shows that x_t can be defined as a suitable linear combination of $x_{t'}$ and new noise. This implies we can also reverse the process to estimate a less noisy state from a more noisy one without adding further stochasticity.

1.1.5 5. Conclusion

Given that x_t is defined as a linear combination of $x_{t'}$ and Gaussian noise, and using the property of the sum of independent Gaussian variables, we conclude that we can achieve intermediate reconstructions at any step without the need to add extra noise to the diffusion process. This simplifies the process and helps improve the model's efficiency.

1.2 Sub-question 2: Conditions for Gaussian Reverse Process

In diffusion models, a key assumption in the reverse path is that the conditional distribution $q(x_{t-1}|x_t, x_0)$ is Gaussian. Under what conditions is this assumption valid, and why does having this distribution as Gaussian allow the reverse path to effectively approximate the distribution $q(x_{t-1}|x_t)$?

1.2.1 Background

1. Forward Process: In diffusion models, the forward process is defined by adding Gaussian noise at each step. In other words:

$$q(x_t|x_{t-1}) = \mathcal{N}(x_t; \sqrt{1 - \beta_t} x_{t-1}, \beta_t I)$$

Since at each step Gaussian noise with a linear mean from the previous step is added, it can be shown that the distribution of x_t given x_0 is also Gaussian:

$$q(x_t|x_0) = \mathcal{N}(x_t; \sqrt{\bar{\alpha}_t} x_0, (1 - \bar{\alpha}_t) I)$$

where $\alpha_t = 1 - \beta_t$ and $\bar{\alpha}_t = \prod_{s=1}^t \alpha_s$.

2. Reverse Conditional Distribution Given Original Data: If we know the original data x_0 , we can derive the conditional distribution $q(x_{t-1}|x_t, x_0)$ using Bayes' theorem. Since all involved distributions ($q(x_t|x_{t-1})$ and $q(x_{t-1}|x_0)$) are Gaussian, the resulting posterior $q(x_{t-1}|x_t, x_0)$ will also be analytically Gaussian:

$$q(x_{t-1}|x_t, x_0) = \mathcal{N}(x_{t-1}; \tilde{\mu}_t(x_t, x_0), \tilde{\beta}_t I)$$

This Gaussian distribution can be derived from the closed-form expressions provided in the original DDPM paper.

1.2.2 Conditions for $q(x_{t-1}|x_t)$ to be Gaussian

Now, let's address the main question. The true reverse distribution $q(x_{t-1}|x_t)$ is actually a mixture distribution over all possible data points x_0 , which can be expressed as:

$$q(x_{t-1}|x_t) = \int q(x_{t-1}|x_t, x_0)q(x_0|x_t)dx_0$$

Since $q(x_0|x_t)$ is itself a complex (and usually non-Gaussian) distribution of the original data, this integral is a mixture of Gaussian distributions $q(x_{t-1}|x_t, x_0)$ with weights derived from $q(x_0|x_t)$. A linear combination of multiple Gaussian distributions with different means and variances is not necessarily a pure Gaussian. This means that if the true data distribution $p(x_0)$ is not Gaussian (which is almost always the case for real-world data like images), then $q(x_{t-1}|x_t)$ will not be a pure Gaussian distribution; rather, it will be a complex Gaussian mixture.

The distribution $q(x_{t-1}|x_t)$ is only guaranteed to be Gaussian if the original data distribution $p(x_0)$ is also Gaussian. This is a very restrictive assumption that does not hold in practice.

1.2.3 Approximating the Reverse Path

Although $q(x_{t-1}|x_t)$ is practically a complex mixture, we know that $q(x_{t-1}|x_t, x_0)$ is a simple, tractable Gaussian. The core idea of diffusion models is to approximate the intractable true reverse distribution $q(x_{t-1}|x_t)$ with a learned Gaussian distribution $p_\theta(x_{t-1}|x_t)$.

$$p_\theta(x_{t-1}|x_t) = \mathcal{N}(x_{t-1}; \mu_\theta(x_t, t), \Sigma_\theta(x_t, t))$$

This approximation is justified because even if the true posterior is a complex mixture, a single Gaussian can be a reasonable approximation, especially when the step sizes β_t are small. When $\beta_t \rightarrow 0$, the change between steps is minimal, and the reverse conditional also approaches a Gaussian.

The model is trained to make $p_\theta(x_{t-1}|x_t)$ as close as possible to the true posterior $q(x_{t-1}|x_t, x_0)$, effectively learning to predict the mean and variance of the denoising step. This is why the Gaussian assumption is central to the feasibility and success of DDPMs.

1.3 Sub-question 3: DDPM Objective Function

In Denoising Diffusion Probabilistic Models (DDPM), the objective function is derived from the Variational Lower Bound (VLB) on the negative log-likelihood of the data. This cost function can be broken down into three main parts:

1. **Kullback-Leibler Divergence at the Final Step (L_T):** This term measures the difference between the prior distribution $p(x_T)$ (usually a standard Gaussian $\mathcal{N}(0, I)$) and the aggregated posterior $q(x_T|x_0)$. Its purpose is to ensure that after the full forward process, the data distribution is transformed into something indistinguishable from pure noise.

$$L_T = D_{KL}(q(x_T|x_0)||p(x_T))$$

2. **Sum of Denoising Matching Terms (L_{t-1} for $t = 2, \dots, T$):** This is the core part of the objective. Each term measures the KL divergence between our learned reverse distribution $p_\theta(x_{t-1}|x_t)$ and the true, but intractable, reverse posterior $q(x_{t-1}|x_t)$. By conditioning on x_0 , this can be expressed using the tractable posterior:

$$L_{t-1} = \mathbb{E}_{q(x_t, x_0)}[D_{KL}(q(x_{t-1}|x_t, x_0)||p_\theta(x_{t-1}|x_t))]$$

This term trains the model to accurately perform one step of denoising.

3. **Reconstruction Term (L_0):** This term is the negative log-likelihood of the original data x_0 being reconstructed from x_1 by the model. It is analogous to the reconstruction term in a VAE.

$$L_0 = -\mathbb{E}_{q(x_1|x_0)}[\log p_\theta(x_0|x_1)]$$

This ensures that the final step of the reverse process generates a high-fidelity version of the original data.

1.3.1 Why are some terms simplified or ignored in practice?

In the DDPM paper, the authors introduce a simplified objective function, L_{simple} , for practical training.

- **Term L_T :** This term is often ignored. With a sufficiently large number of steps T and a well-designed noise schedule, the distribution $q(x_T|x_0)$ will naturally become very close to the standard Gaussian prior $\mathcal{N}(0, I)$. Therefore, the KL divergence becomes negligible and does not provide a useful learning signal.
- **Term L_0 and L_{t-1} Simplification:** The authors show that the terms L_{t-1} (for $t > 1$) can be re-written to directly minimize the difference between the true noise ϵ_t added at step t and the noise $\epsilon_\theta(x_t, t)$ predicted by the neural network. This leads to a much simpler objective:

$$L_{simple}(\theta) = \mathbb{E}_{t, x_0, \epsilon} [\|\epsilon - \epsilon_\theta(\sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon, t)\|^2]$$

This objective function is empirically found to be easier to optimize and yields better results. While it seems to ignore the reconstruction term L_0 , by training the model to accurately predict noise at all timesteps (including small t), it implicitly learns to perform the final reconstruction step effectively. The weighting of terms in the full VLB objective can be complex, whereas the simplified objective treats all noise prediction tasks equally, which works very well in practice.

1.3.2 Conclusion

In practice, the main focus of DDPM training is on the simplified noise prediction objective (L_{simple}). The term L_T is ignored as it provides little to no gradient, and the complex KL divergence terms are replaced by a straightforward Mean Squared Error loss on the predicted noise. This simplification not only makes training more stable and efficient but has also been shown empirically to produce higher-quality samples.

1.4 Sub-question 4: Conditions for DDIM and DDPM Equivalence

The DDIM (Denoising Diffusion Implicit Models) model is a generalization of the DDPM model, designed to allow for a more flexible and faster, non-Markovian reverse process.

1.4.1 Mathematical Explanation

Forward Process (Same for both) The forward process is identical for both models:

$$q(x_t|x_0) = \mathcal{N}(x_t; \sqrt{\bar{\alpha}_t}x_0, (1 - \bar{\alpha}_t)I)$$

Reverse Process in DDPM The DDPM reverse process is stochastic and Markovian. The update step is:

$$x_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_\theta(x_t, t) \right) + \sigma_t z$$

where $z \sim \mathcal{N}(0, I)$ and $\sigma_t^2 = \tilde{\beta}_t = \frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t} \beta_t$.

Reverse Process in DDIM The DDIM reverse process is more general. It predicts x_0 first and then uses it to step to x_{t-1} . The update is:

$$x_{t-1} = \sqrt{\bar{\alpha}_{t-1}} \hat{x}_0 + \sqrt{1 - \bar{\alpha}_{t-1} - \sigma_t^2} \cdot \hat{\epsilon}_t + \sigma_t z$$

where $\hat{x}_0 = \frac{x_t - \sqrt{1 - \bar{\alpha}_t} \epsilon_\theta(x_t, t)}{\sqrt{\bar{\alpha}_t}}$ is the predicted clean image, and $\hat{\epsilon}_t$ is the direction pointing from x_t to $\hat{x}_0$. The amount of stochasticity is controlled by σ_t , which is parameterized by a new variable η :

$$\sigma_t^2 = \eta \tilde{\beta}_t$$

When $\eta = 0$, the process is deterministic (implicit). When $\eta = 1$, the process is fully stochastic.

1.4.2 Conditions for Equivalence

The DDIM model becomes exactly equivalent to the DDPM model when the parameter $\eta = 1$ is set. In this case:

1. **Stochasticity Matches:** With $\eta = 1$, the variance of the added noise in the DDIM step, σ_t^2 , becomes exactly equal to the variance of the DDPM reverse conditional, $\tilde{\beta}_t$.

2. **Markovian Process:** The specific choice of $\eta = 1$ makes the DDIM generative process Markovian, just like DDPM. The non-Markovian flexibility of DDIM is removed.
3. **Identical Update Step:** With $\eta = 1$, the mathematical formula for the DDIM update step simplifies to be identical to the DDPM update step.

Therefore, DDPM is a specific instance of the more general DDIM framework, corresponding to the case with maximum stochasticity ($\eta = 1$).

1.5 Sub-question 5: Early-Stopped DDPM (ES-DDPM)

The ES-DDPM paper aims to accelerate the slow sampling process of standard DDPMs, which can take thousands of steps.

1.5.1 The Core Idea

The approach is to stop the forward diffusion process "early," at an intermediate timestep $T_0 \ll T$. At this point, the data distribution $q(x_{T_0}|x_0)$ is noisy but has not yet fully collapsed into a simple Gaussian. It retains significant structure from the original data, making it a complex, non-Gaussian distribution. Sampling directly from this distribution is difficult.

1.5.2 The Proposed Solution

The solution is a two-stage process that uses a pre-trained, fast generative model (like a VAE or GAN) as a starting point.

1. **Part 1 (Initial Generator):** First, sample a low-quality but structurally plausible image from a fast generator like a VAE. This provides a good "guess" or starting point that is already on the data manifold.
2. **Part 2 (ES-DDPM Refinement):** Treat the generated image as a "clean" image and apply the DDPM forward process for only T_0 steps to noise it. Then, use a DDPM model (which only needs to be effective up to step T_0) to run the reverse process for T_0 steps. This refines the initial guess, adding high-frequency details and improving overall quality.

Essentially, the fast generator covers the long-range dependencies and overall structure, while the short-run diffusion model acts as a powerful refiner.

1.5.3 Advantages of this Method

- **Significant Speed-up:** The number of required diffusion steps is drastically reduced from T (e.g., 1000) to T_0 (e.g., 100-200), leading to a massive reduction in sampling time.
- **Improved Sample Quality:** By leveraging the strengths of two different types of models, the final sample quality can sometimes exceed that of the original DDPM.
- **Modularity:** This method is modular. It allows combining pre-trained models without needing to retrain them from scratch. A pre-trained VAE/GAN can be paired with a pre-trained DDPM.

- **Compatibility:** It can be combined with other DDPM acceleration techniques (like step jumping in DDIM) for even greater speed-ups.

Part II

Implementation Questions

2 DDPM and DDIM Implementation Analysis

2.1 Visual Evolution Through Epochs

We begin by observing the model's learning progress by visualizing generated samples at different stages of training.

2.1.1 Epoch 2

At this early stage, the model has barely begun to learn the data distribution. The outputs are noisy and lack coherent structure, which is expected.

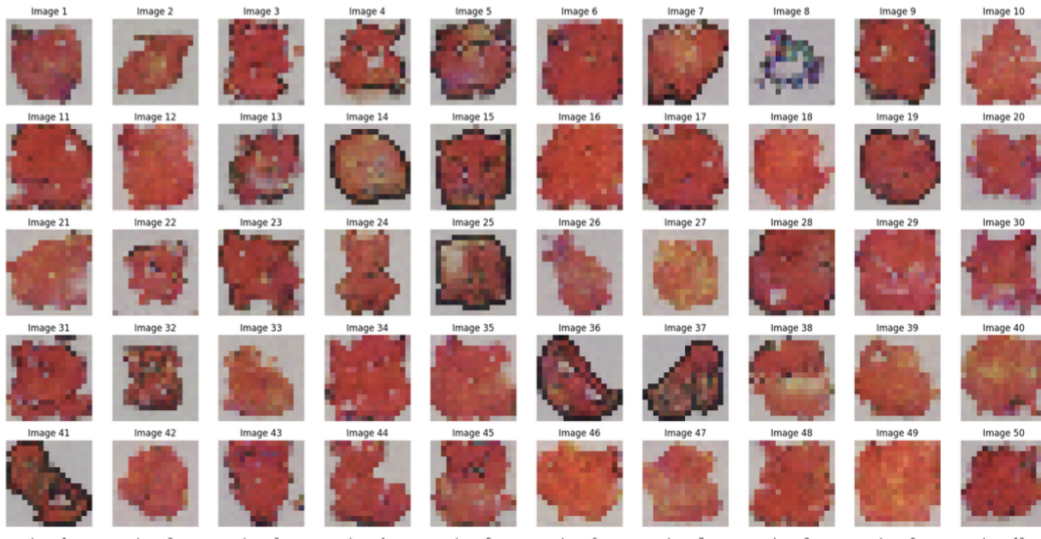


Figure 1: Figure 1.1: Images generated at epoch 2.

2.1.2 Epoch 20

After 20 epochs, significant learning has occurred. The model is now generating recognizable shapes and structures that resemble the training data, although they still lack fine detail and contain artifacts.

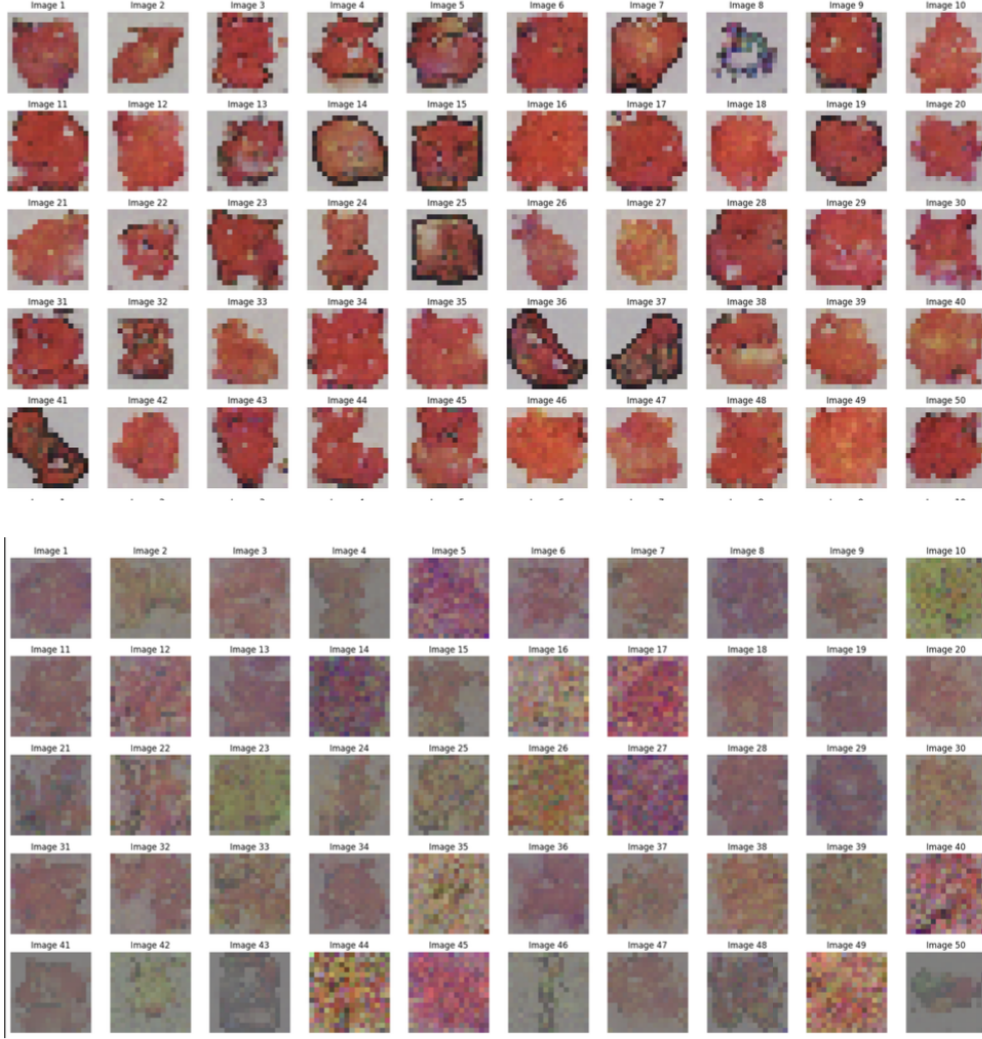


Figure 2: Figure 1.2: Images generated at epoch 20.

2.1.3 Epoch 40 (Final)

By the final epoch, the model produces much higher quality samples. The images shown below compare the outputs of the standard DDPM (top row) and the faster DDIM (bottom row). We observe that while DDIM significantly accelerates generation, it comes at the cost of a noticeable drop in image quality and diversity. This trade-off is central to the practical application of these models.

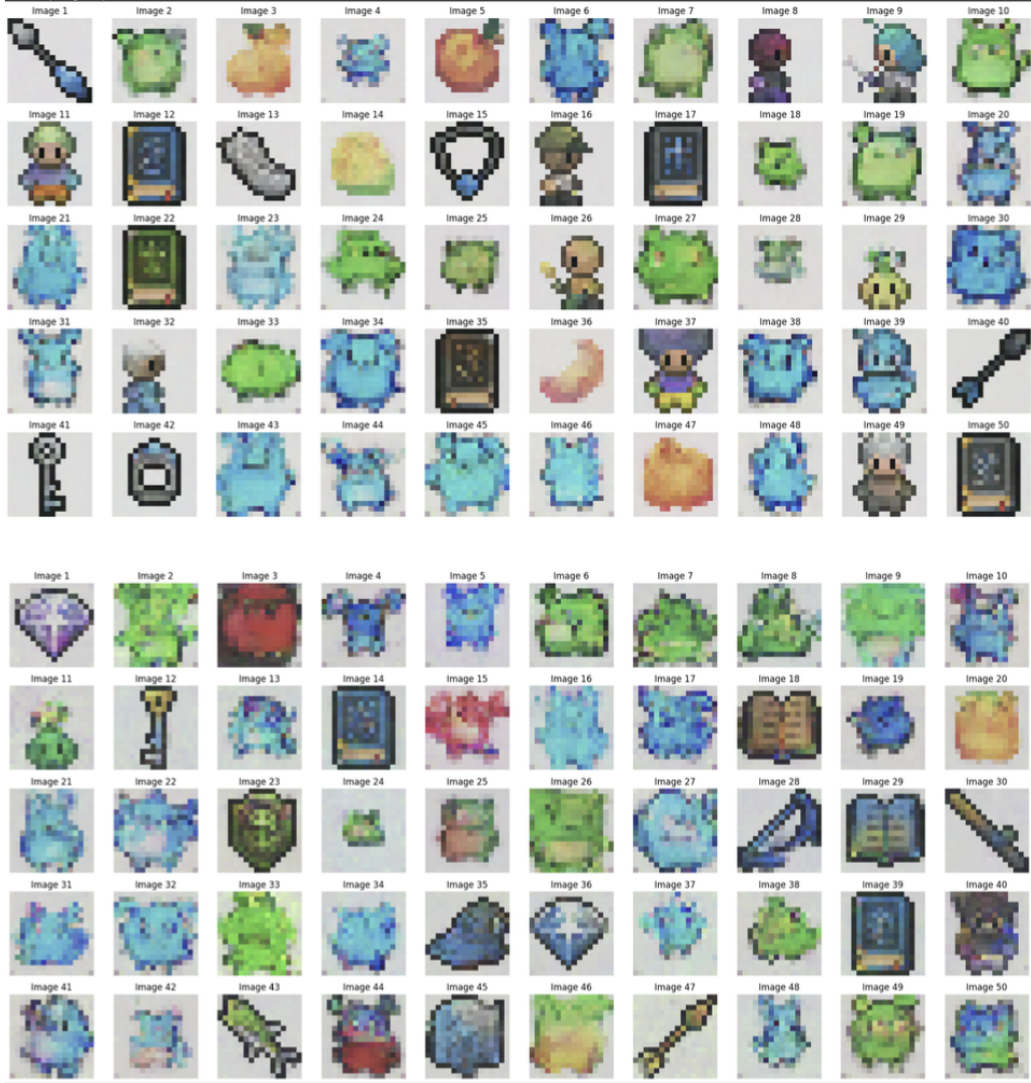


Figure 3: Figure 1.3: Images generated at epoch 40 (Top: DDPM, Bottom: DDIM).

2.2 Final Generated Samples

Here we present a larger batch of final samples from both the DDPM and DDIM models to better assess their final performance.

2.2.1 DDPM Final Samples

The DDPM model, after full training, produces diverse and high-quality samples that are faithful to the training data distribution.



Figure 4: Figure 1.4: Final samples generated by DDPM.

2.2.2 DDIM Final Samples

The DDIM samples are generated much faster but exhibit less detail and variety compared to the DDPM samples.

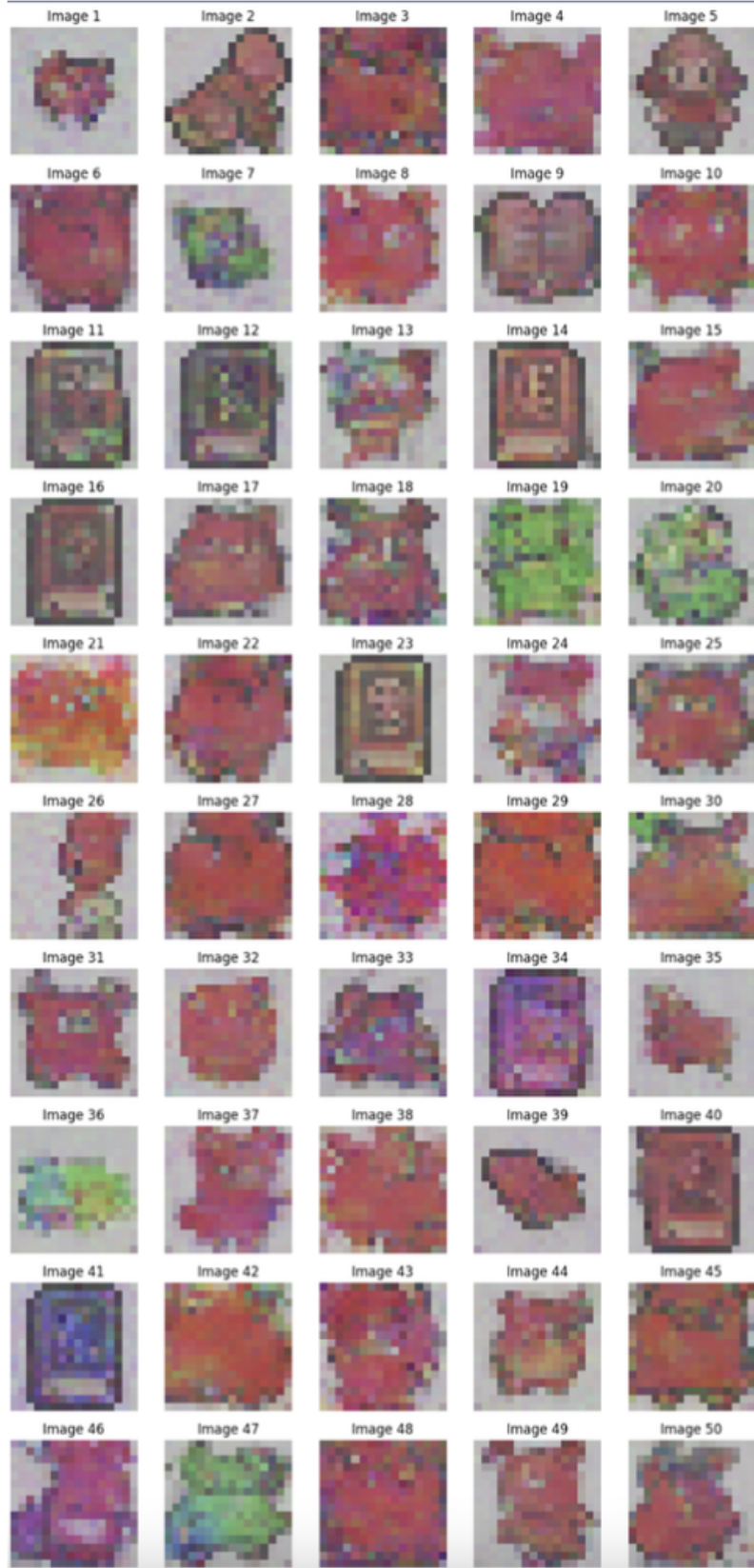


Figure 5: Figure 1.5: Final samples generated by DDIM.

2.2.3 DDIM Deterministic Samples ($\eta = 0$)

When DDIM is run in its fully deterministic mode ($\eta = 0$), it produces a single, fixed output for a given starting noise vector. The quality is comparable to the stochastic DDIM but lacks any variation.

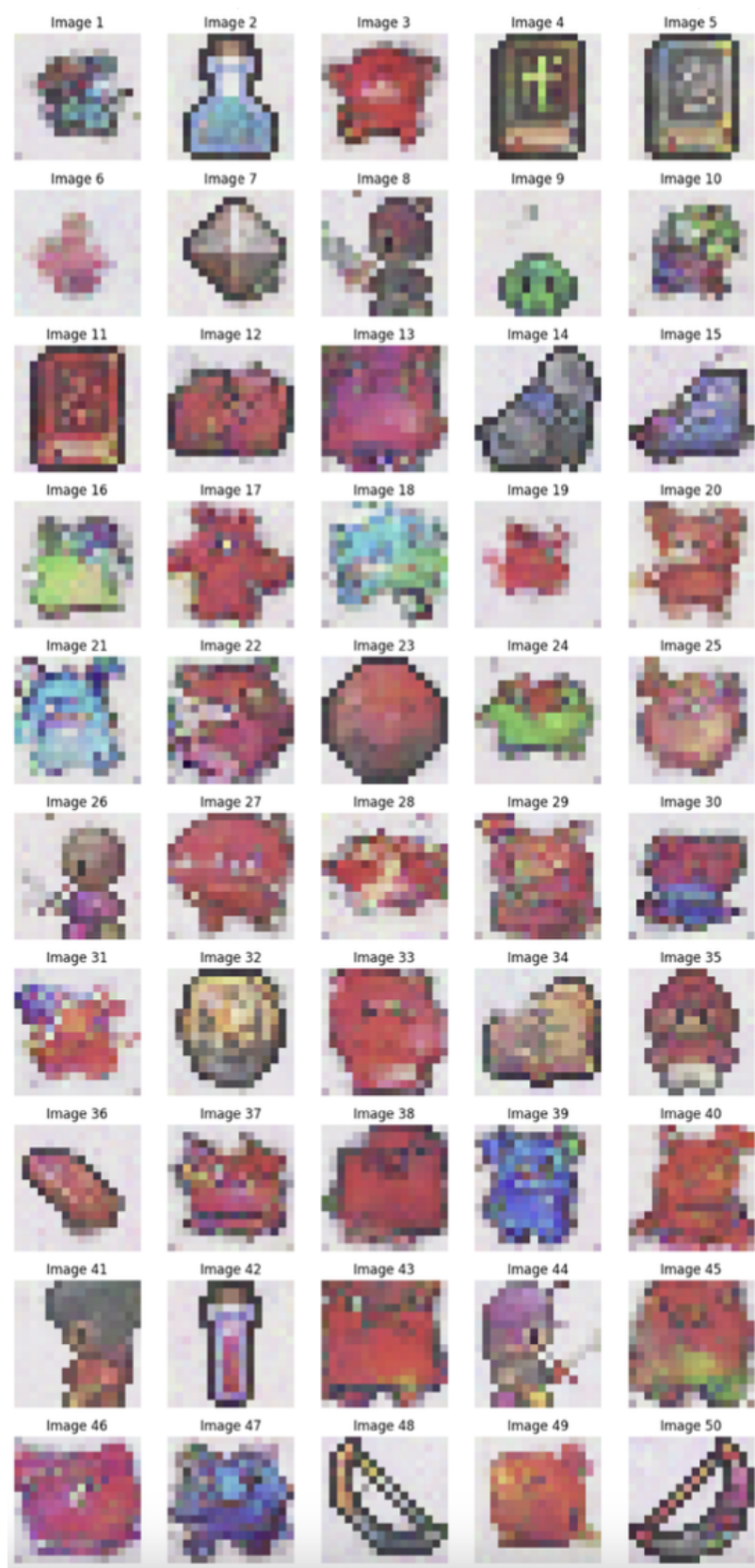


Figure 6: Figure 1.6: Samples generated by DDIM in deterministic mode.

2.3 Training Analysis and Diagnostics

```
1 print("\nGenerating final samples using DDPM and DDIM...")
2 final_samples_ddpm = sample_ddpm(nn_model, n_sample=50, ...).cpu()
3 final_samples_ddim = sample_ddim(nn_model, n_sample=50, ..., eta=0,
  n_steps=400).cpu()
4 plot_sample(final_samples_ddpm, 50, n_rows=10)
5 plt.savefig(os.path.join(final_ddpm_dir, 'ddpm_final_samples.png'))
```

Listing 1: Code snippet for final sample generation.

Figure 7: Figure 1.7: Code snippet showing parameters for DDIM sampling. Found on page 22.

A key challenge observed during experimentation was that the DDIM model required a sufficient number of training iterations (empirically ≥ 300) to produce meaningful output. With fewer iterations, the model would fail to learn the denoising process, and its output would be indistinguishable from random noise. This highlights the model's need for adequate training time to capture the data's underlying structure.

2.3.1 Loss and Quality Analysis

The reduction in the training loss over time indicates that the model is successfully learning to predict and remove noise.

- **Reduction in Loss:** A consistently decreasing loss curve confirms that the model is improving its performance on the training objective.
- **Potential for Improvement:** While the final loss is low, allocating more training time and computational resources would likely lead to even lower loss values and, consequently, higher-quality image reconstructions with finer details.
- **Quality of DDPM Images:** The quality of the final DDPM images suggests that the achieved loss value is sufficient for generating visually appealing results that capture the essence of the data distribution.
- **Time vs. Quality Trade-off:** There is an inherent trade-off. While more training could improve quality, it comes at a higher computational cost. The current results represent a reasonable balance, achieving acceptable quality within a limited training budget.

2.3.2 Loss and FID Charts

The training loss curve shows a smooth and steady decrease, indicating a stable training process.

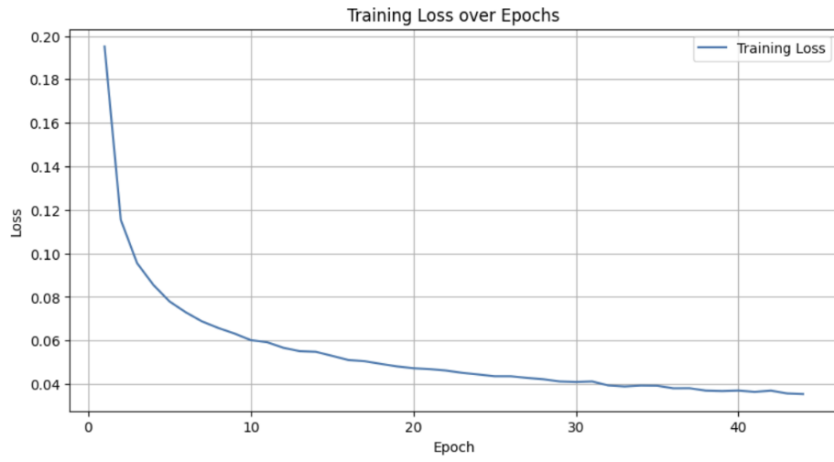


Figure 8: Figure 1.8: DDPM Training Loss over Epochs.

The Fréchet Inception Distance (FID) score measures the quality and diversity of generated images compared to real images. Lower is better.

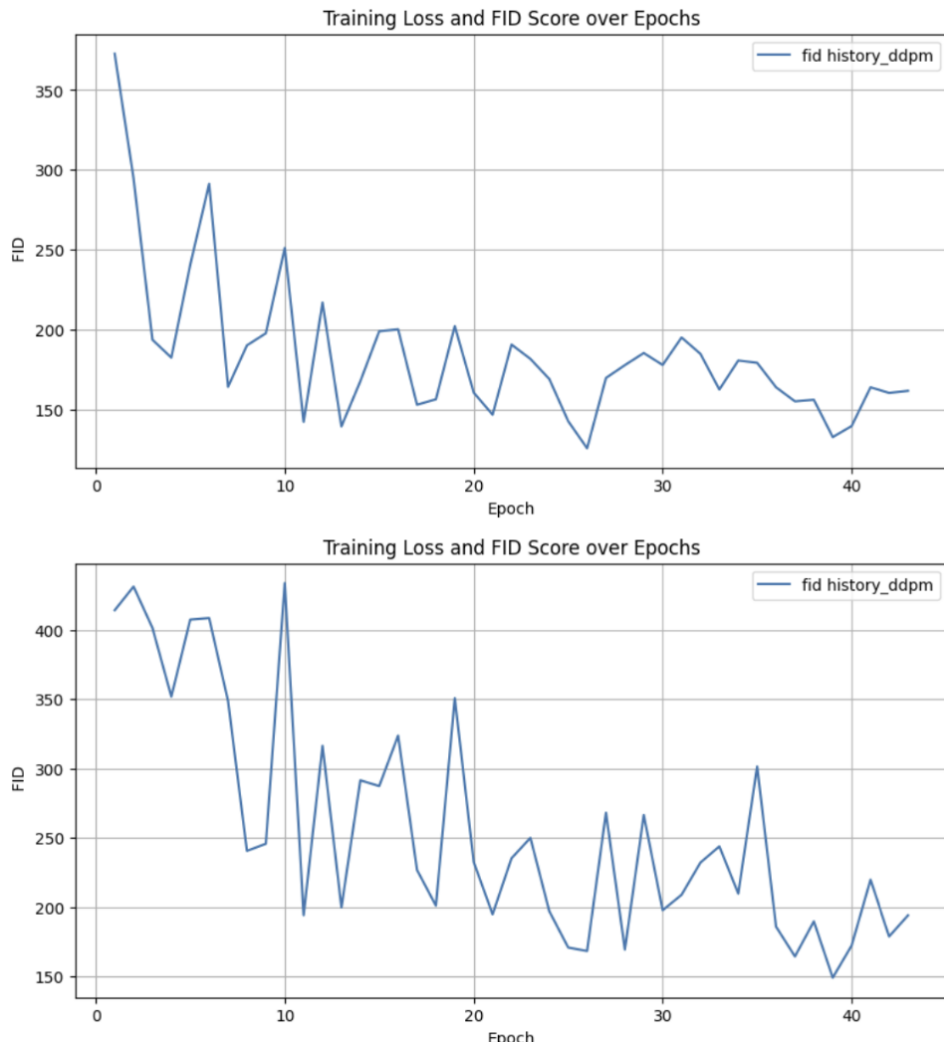


Figure 9: Figure 1.9: FID Score over Epochs for DDPM (top) and DDIM (bottom).

2.3.3 FID Chart Analysis

1. DDPM (Top Chart):

- **Start of Training:** The initial FID is very high (350), as expected from an untrained model.
- **Downward Trend:** The FID score decreases steadily, showing consistent improvement in image quality.
- **Stability:** Towards the end of training, the FID score stabilizes around 150-175, indicating that the model has converged to a good performance level.

2. DDIM (Bottom Chart):

- **Start of Training:** The initial FID is even higher than DDPM's (400), suggesting DDIM struggles more in the early stages of training.
- **Downward Trend:** The FID also decreases but with significantly more volatility and fluctuation than DDPM. This may be due to its deterministic nature, which can make it harder to consistently match the true data distribution.
- **Stability:** The final FID score is comparable to DDPM's, but it remains more unstable, with larger fluctuations even at the end of training.

3. Comparison and Conclusion: DDPM demonstrates a more stable and robust learning trajectory, achieving better quality more consistently throughout training. DDIM, while ultimately reaching a similar performance level, shows more instability. This makes DDPM a better choice for applications prioritizing final quality and reliability, whereas DDIM is preferable when sampling speed is the primary concern, and some quality/stability trade-off is acceptable.

2.4 Question 6: DDIM Parameter Ablation Study

We investigate the impact of DDIM's parameters ('eta' and 'n_steps') on the speed – quality trade-off.

2.4.1 Case 1: Fast Sampling ('eta=0.0', 'n_steps = 400')

This configuration uses the deterministic DDIM with fewer steps to maximize speed.

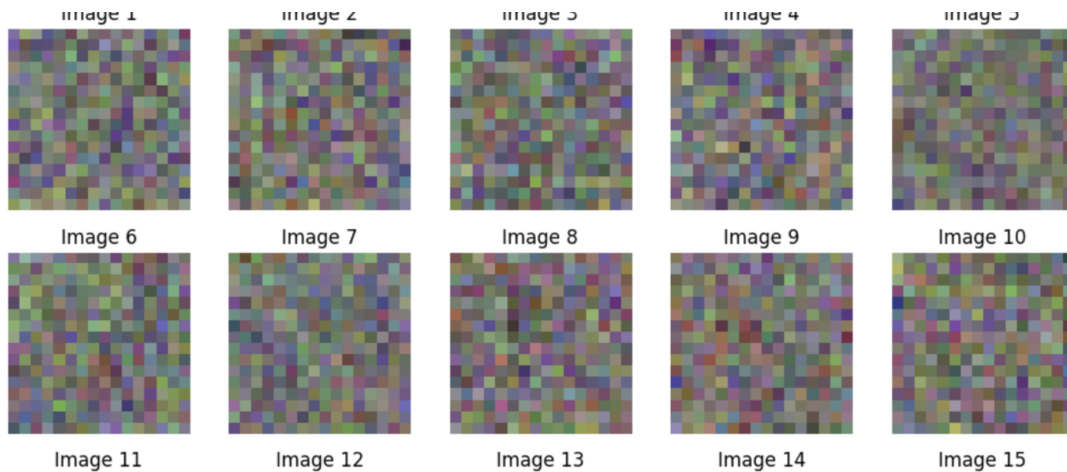


Figure 10: DDIM with 'eta=0.0'. DDPM Time: 6.68s, DDIM Time: 3.25s.

Analysis: The generation time is more than halved compared to DDPM. However, this comes with a significant and visible degradation in image quality. This setting is suitable for applications where speed is critical and lower fidelity is acceptable.

2.4.2 Case 2: Quality-Focused Sampling (‘eta=1’, ‘n_steps = 600’)

This configuration uses more steps and a stochastic setting (‘eta=1’) to better preserve quality.

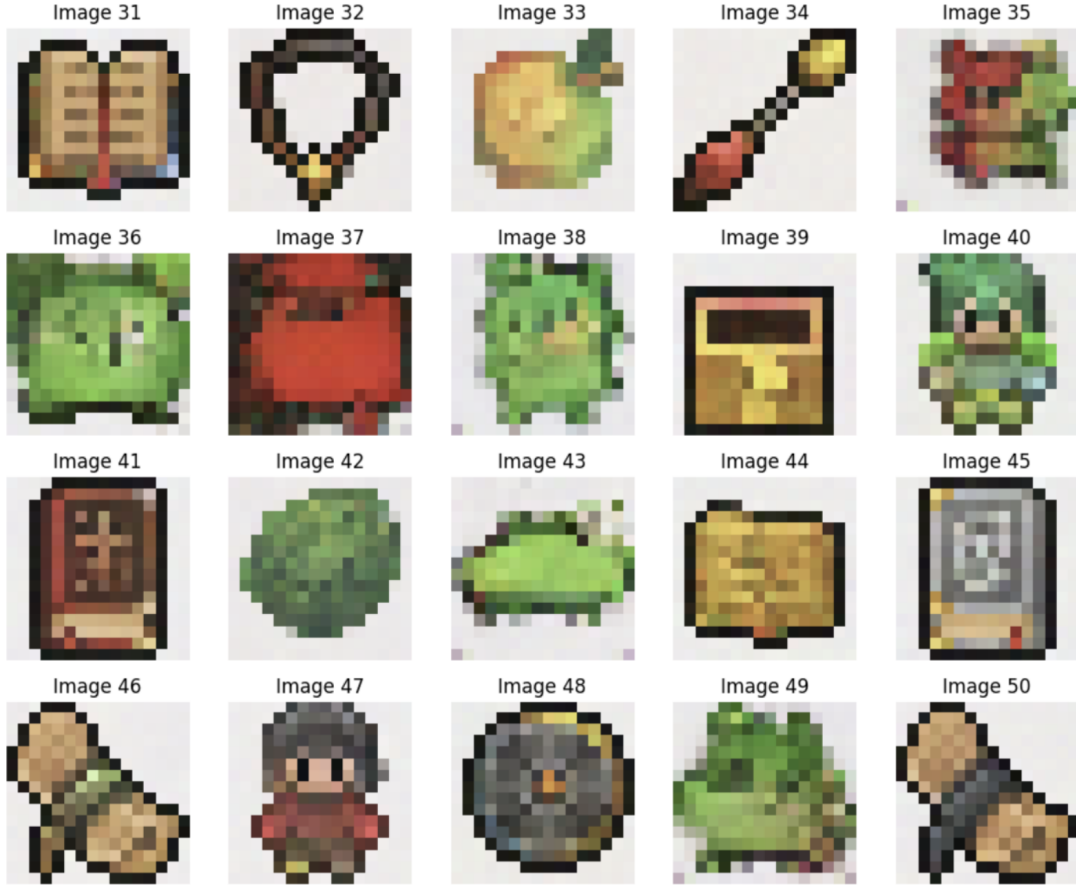


Figure 11: Figure 1.11: DDIM with ‘eta=1’. DDPM Time: 7.09s, DDIM Time: 5.61s.

Analysis: The generation time is only slightly faster than DDPM, but the image quality is almost perfectly preserved. This setting provides a good balance, offering a modest speed-up without a major sacrifice in quality, making it suitable for general-purpose use.

2.5 Question 4: The Role of Noise ‘z’ in DDPM

In the DDPM algorithm, the noise vector z plays a crucial role during the reverse sampling process.

2.5.1 Role of Noise ‘z’ in the Sampling Process

The goal of DDPM sampling is to start from pure Gaussian noise x_T and iteratively denoise it to obtain a clean sample x_0 . The random noise $z \sim \mathcal{N}(0, I)$ is added at each step of this reverse process. This serves two key functions:

1. **Maintaining Diversity:** Without the addition of stochastic noise z at each step, the reverse process would be deterministic. This would mean that a single starting noise map x_T would always produce the exact same output image x_0 . The noise z ensures that the path taken is stochastic, allowing the model to generate a diverse range of different images from different noise vectors.
2. **Correcting the Distribution:** The forward process is stochastic. To accurately model the reverse of this process, the reverse step must also be stochastic. The noise term z ensures that the variance of the learned reverse step $p_\theta(x_{t-1}|x_t)$ matches the true posterior conditional $q(x_{t-1}|x_t, x_0)$.

2.5.2 Why is Noise ‘z’ Zero at t=0?

The final step of the reverse process is from x_1 to x_0 . At this stage, the goal is to produce the final, clean, and deterministic output.

1. **No Need for Stochasticity:** The purpose of the noise was to explore the distribution space. In the final step, we are no longer exploring but reconstructing the single best estimate of the clean image.
2. **Accurate Reconstruction:** Adding noise at the very last step would degrade the final image, introducing unwanted artifacts and preventing a clean reconstruction.
3. **Consistency with the Process:** The reverse process is designed to gradually *remove* noise. The final step should complete this removal, not add more noise back in. Therefore, setting $z = 0$ at $t = 0$ (or, more accurately, not having a noise term in the final x_0 prediction) is a logical conclusion of the process.

3 Score-Based Models: Theory and Implementation

3.1 Theoretical Foundations

3.1.1 Sub-question 1: The Partition Function Problem

In many probabilistic models, the probability density is defined via an energy function $f_\theta(x)$:

$$p_\theta(x) = \frac{e^{-f_\theta(x)}}{Z_\theta}$$

The main challenge is the normalization constant, or **partition function**, $Z_\theta = \int e^{-f_\theta(x)} dx$. For high-dimensional data, this integral is intractable to compute. This prevents direct evaluation of the likelihood and makes training via maximum likelihood optimization impossible.

Solution: Score-based models circumvent this by learning the **score function**, which is the gradient of the log-probability with respect to the data:

$$s_\theta(x) = \nabla_x \log p_\theta(x) = \nabla_x (-f_\theta(x) - \log Z_\theta) = -\nabla_x f_\theta(x)$$

Since Z_θ does not depend on x , its gradient is zero. This means we can learn the score function by only learning the gradient of the energy function, completely bypassing the need to compute the intractable partition function.

3.1.2 Sub-question 2: Sliced Score Matching (SSM)

The classic Score Matching objective requires computing the trace of the Hessian matrix of the log-density, which is computationally very expensive (scaling with the data dimensionality). Sliced Score Matching (SSM) provides an efficient, unbiased estimate of this objective. Instead of computing the full Hessian, SSM projects the score vectors onto random 1D directions and computes a simpler objective. It relies on the identity:

$$\text{tr}(\nabla_x s_\theta(x)) = \mathbb{E}_v[v^T (\nabla_x s_\theta(x)) v]$$

where v is a random vector with a zero mean and identity covariance. This reduces the expensive trace computation to a series of vector-Jacobian products, which can be calculated efficiently with automatic differentiation, making score matching feasible for deep models and high-dimensional data.

3.1.3 Sub-question 3: Denoising Score Matching (DSM)

DSM is another technique to make score matching tractable. The core idea is to perturb the data with Gaussian noise and then train the model to learn the score of this noised data distribution, which is an easier target. The objective for DSM is:

$$L_{DSM}(\theta) = \mathbb{E}_{p_{data}(x)} \mathbb{E}_{q_\sigma(\tilde{x}|x)} [\|s_\theta(\tilde{x}) - \nabla_{\tilde{x}} \log q_\sigma(\tilde{x}|x)\|^2]$$

The key insight is that the target score, $\nabla_{\tilde{x}} \log q_\sigma(\tilde{x}|x)$, is analytically tractable and simple to compute for a Gaussian noise distribution $q_\sigma(\tilde{x}|x) = \mathcal{N}(\tilde{x}; x, \sigma^2 I)$:

$$\nabla_{\tilde{x}} \log q_\sigma(\tilde{x}|x) = -\frac{1}{\sigma^2}(\tilde{x} - x)$$

This transforms the difficult problem of learning the score of the complex data distribution into the simpler problem of learning to denoise the data.

3.1.4 Sub-question 4: Langevin Dynamics Sampling

Once a score model $s_\theta(x) \approx \nabla_x \log p_{\text{data}}(x)$ is trained, we need a way to draw samples from the underlying distribution $p_{\text{data}}(x)$. Since we only have the gradient (score), not the density itself, we cannot use standard sampling methods. **Langevin Dynamics** is an iterative MCMC method that uses the score to generate samples. Starting from a random point x_0 , it iteratively updates the sample using the rule:

$$x_{t+1} = x_t + \frac{\epsilon^2}{2} s_\theta(x_t) + \epsilon z_t$$

where $z_t \sim \mathcal{N}(0, I)$ is random noise and ϵ is the step size. The score term $s_\theta(x_t)$ pushes the sample towards regions of higher probability density (modes), while the noise term z_t allows for exploration of the space. After many iterations, the distribution of x_t converges to the target data distribution.

3.1.5 Sub-question 5: Annealed Langevin Dynamics

Standard Langevin Dynamics can fail for two reasons:

1. Data often lies on a low-dimensional manifold, where the score may be undefined or difficult to learn in the ambient high-dimensional space.
2. If the data distribution has multiple disconnected modes separated by low-probability regions, the sampler can get "stuck" in one mode and fail to explore the full distribution.

Annealed Langevin Dynamics solves this by using a sequence of noise levels $\sigma_1 > \sigma_2 > \dots > \sigma_L$. The process starts by sampling from the heavily noised distribution $p_{\sigma_1}(x)$, where the added noise "fills in" the low-density regions and smooths out the distribution, making it easy to sample from. The samples are then gradually refined by running Langevin dynamics at progressively smaller noise levels, using the samples from the previous level as initialization. This "annealing" process allows the sampler to explore the global structure of the distribution at high noise levels and then refine the details at low noise levels, effectively finding all modes and generating high-quality samples.

3.2 Implementation and Analysis

3.2.1 Data Visualization

We first visualize the target distribution, which is a mixture of two Gaussians.

```
1 def gaussian_pdf(x, mean, cov):
2     # ... implementation ...
3 def p_x(points, params):
4     pdf1 = params['w1'] * gaussian_pdf(points, params['mean1'], params[
5         'cov1'])
6     pdf2 = params['w2'] * gaussian_pdf(points, params['mean2'], params[
7         'cov2'])
8     return pdf1 + pdf2
9 \end{lstlisting}
10 \begin{figure}[H]
11     \centering
12     \includegraphics[width=0.6\textwidth]{page47_heatmap.png}
13     \caption{Figure 2.1: Heatmap of the target probability density.}
14 \end{figure}
```

```

13 \begin{figure}[H]
14     \centering
15     \includegraphics[width=0.6\textwidth]{page47_scatter.png}
16     \caption{Figure 2.2: Scatter plot of training and test samples
17         drawn from the distribution.}
18 \end{figure}
19 \subsubsection{ScoreNet Model and Training}
20 We use a simple MLP to model the score function.
21 \begin{lstlisting}[language=Python, caption={ScoreNet model
22     architecture.}]
23 class ScoreNet(nn.Module):
24     """Neural network to estimate the score function."""
25     def __init__(self, input_dim=3, hidden_dim=256, num_layers=6):
26         # ... MLP architecture ...
27     def forward(self, x, sigma):
28         x_sigma = torch.cat([x, sigma], dim=1)
29         return self.net(x_sigma)

```

Listing 2: Code for Gaussian mixture PDF.

The model is trained using the Denoising Score Matching loss.

```

1 def score_matching_loss(x, x_noisy, epsilon, sigma, outputs):
2     """Compute the score matching loss."""
3     term = outputs + epsilon / (sigma ** 2)
4     loss = 0.5 * torch.mean((term) ** 2)
5     return loss

```

Listing 3: DSM loss function.

3.2.2 Analysis of Score Fields with Fixed Sigma

We train separate models for fixed noise levels $\sigma \in \{1, 3, 7\}$ and visualize the learned score fields.

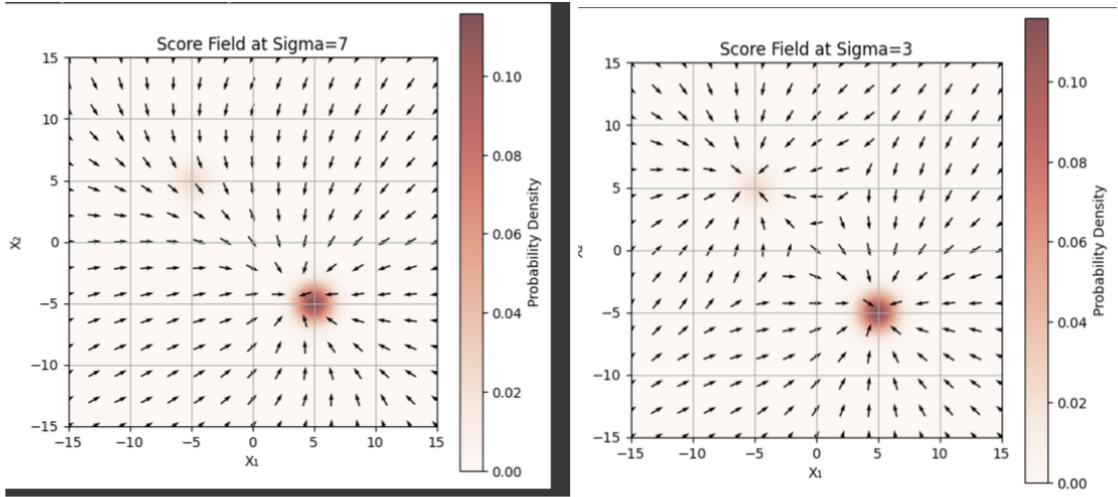
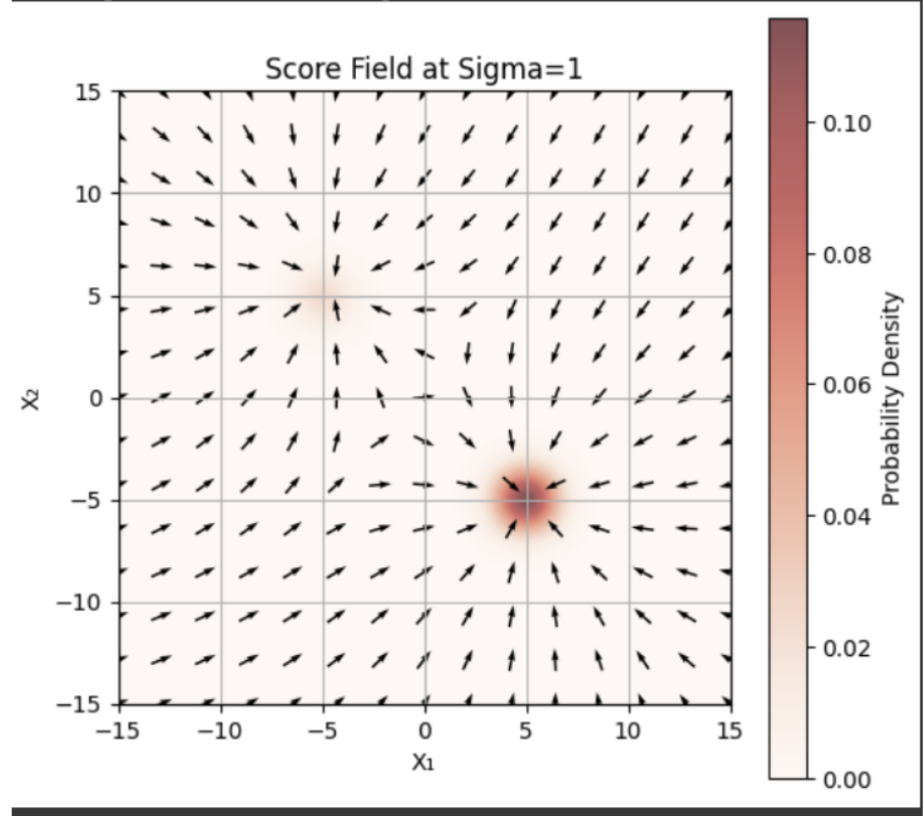


Figure 12: Figure 2.3: Learned score fields for $\sigma = 1$ (top), $\sigma = 7$ (bottom left), and $\sigma = 3$ (bottom right).

Analysis:

- **Sigma=1:** The score field is sharp and points strongly towards the two modes. However, the influence of each mode is localized.
- **Sigma=3:** The field is smoother, and the vectors have a wider range of influence.
- **Sigma=7:** The field is very smooth. The two modes have effectively merged into

a single, wider distribution. The score vectors point towards the global center of the data, losing the fine-grained bimodal structure.

This shows that a single fixed sigma is insufficient: small sigma captures details but fails globally, while large sigma captures the global structure but misses details. This motivates the need for annealed sampling across multiple sigmas.

3.2.3 Analysis of Sampling Trajectories

We compare deterministic sampling (following only the score) with stochastic Langevin sampling.

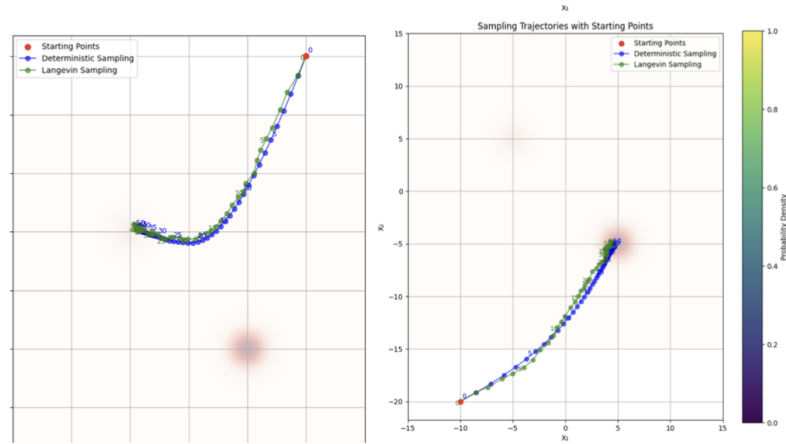


Figure 13: Figure 2.4: Comparison of sampling trajectories.

Analysis:

- **Deterministic Sampling:** The trajectories are smooth and follow the gradient field directly to the nearest mode. It is fast and predictable but lacks exploration.
- **Langevin Sampling:** The trajectories are noisy and stochastic. The added noise allows them to explore the space more broadly and prevents them from collapsing to a single path. This is crucial for generating diverse samples.

```

1 def deterministic_sampling(model, start_points, ...):
2     # ... update without noise ...
3     current_points = current_points + step_size * grad
4
5 def langevin_sampling(model, start_points, ...):
6     # ... update with noise ...
7     noise = torch.randn_like(current_points) * np.sqrt(step_size)
8     current_points = current_points + step_size * grad + noise

```

Listing 4: Deterministic and Langevin sampling functions.

3.2.4 Final Sampling Results and Annealing

We compare the final samples generated by different methods.

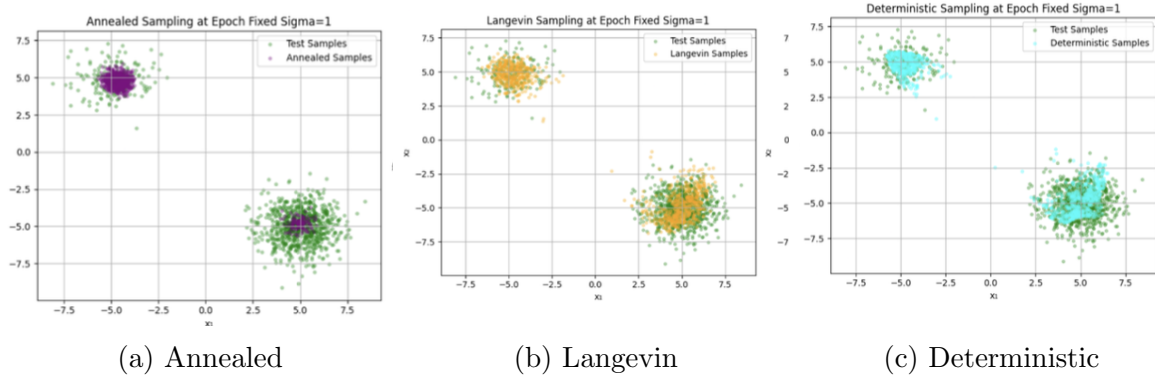


Figure 14: Figure 2.5: Final samples with fixed $\sigma = 1$.

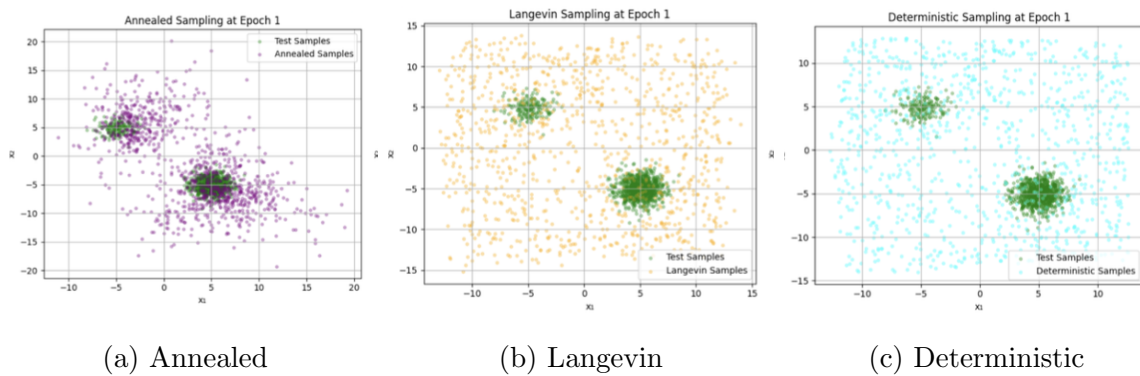


Figure 15: Figure 2.6: Final samples with fixed $\sigma = 7$. The model fails to capture the bimodal structure.

Analysis of Fixed Sigma: Models trained with a single fixed sigma struggle to correctly represent the probability mass of the two modes. They tend to distribute samples somewhat evenly, failing to recognize that one mode should be denser than the other.

3.2.5 Annealed Sampling Implementation and Results

```

1 def annealed_sampling(model, start_points, sigma_start=20, sigma_end=1,
2   ...):
3     """Perform annealed Langevin sampling across a schedule of sigmas."""
4     sigma_values = np.linspace(sigma_start, sigma_end, num_sigmas)
5     # ... loop over sigmas, performing Langevin updates at each ...

```

Listing 5: Annealed Langevin sampling function.

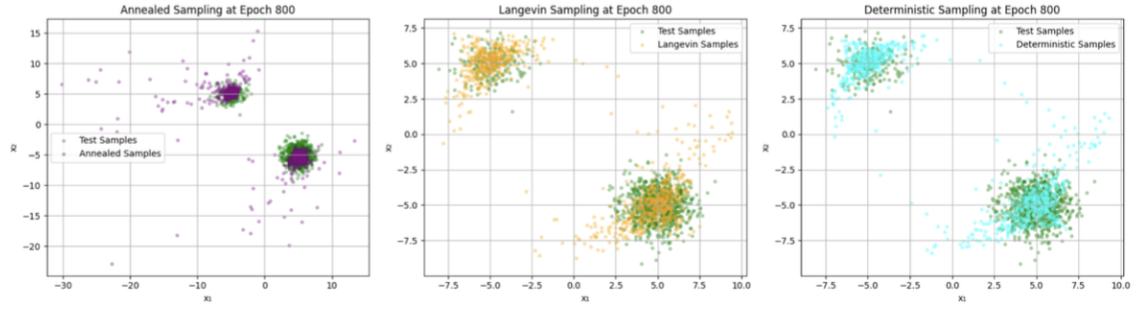


Figure 16: Figure 2.7: Final comparison. Annealed sampling (left) correctly captures the different densities of the two modes, while Langevin (middle) and Deterministic (right) do not.

Final Conclusion: The results clearly demonstrate the superiority of **Annealed Langevin Sampling**. While deterministic and standard Langevin sampling can find the modes of the distribution, they fail to correctly represent the probability density within those modes. The annealed approach, by starting with a high noise level to find the global structure and gradually reducing the noise to refine the details, is the only method that successfully generates samples reflecting the true underlying data distribution, with more samples correctly placed in the higher-probability mode.

References

- [1] Goodfellow, I., et al. (2014). Generative Adversarial Nets. *Advances in Neural Information Processing Systems*, 27, 2672-2680.
- [2] Salimans, T., et al. (2016). Improved Techniques for Training GANs. *Advances in Neural Information Processing Systems*, 29, 2234-2242.
- [3] Arjovsky, M., Chintala, S., & Bottou, L. (2017). Wasserstein GAN. *arXiv preprint arXiv:1701.07875*.
- [4] Behrmann, J., Grathwohl, W., Chen, R. T. Q., Duvenaud, D., Jacobsen, J.-H. (2019). Invertible Residual Networks. *International Conference on Machine Learning (ICML)*.
- [5] Chen, R. T. Q., Behrmann, J., Duvenaud, D., Jacobsen, J.-H. (2019). Residual Flows for Invertible Generative Modeling. *Advances in Neural Information Processing Systems (NeurIPS)*.
- [6] Hutchinson, M. F. (1990). A Stochastic Estimator of the Trace of the Influence Matrix for Laplacian Smoothing Splines. *Communications in Statistics - Simulation and Computation*.
- [7] OpenAI. (2023). ChatGPT.